

1 Description of Software Architecture

The AUV’s software stack runs on **ROS 2 Jazzy**. The workspace is split into ten packages under `auv_ws/src/` – `auv_vision`, `auv_planner`, `auv_telemetry`, `auv_description`, `auv_bringup`, plus four packages currently scaffolded but not yet implemented (`auv_controls`, `auv_propulsion`, `auv_sensors`, `auv_msgs` – each holds only a `CMakeLists.txt/package.xml`, no source) and `auv_teleop` (also an empty stub). Every inter-node link is an asynchronous ROS 2 topic.

```
auv_ws/src/
|-- auv_bringup/          launch/ (sim_mission.launch.py, sim_post_gate.launch.py)
|                         config/ (sim_params.yaml, real_params.yaml)
|-- auv_description/      urdf/, meshes/, models/auv_box/model.sdf, launch/bridge.launch.py
|-- auv_vision/
|   |-- gate_detector_node.py      |-- blue_bin_cv_detector_node.py
|   |-- gate_localizer_node.py     |-- blue_bin_localizer_node.py
|   |-- blue_bin_detector_node.py   |-- green_detector_node.py
|   |-- green_localizer_node.py     |-- underwater_view_node.py
|   '-- bottom_cam_altitude_test.py (test-harness script, not a mission node)
|-- auv_planner/
|   |-- gate_navigator_v2_node.py   (launched -- one-pass gate crossing)
|   |-- gate_navigator_node.py     (legacy round-trip FSM, kept but unused)
|   '-- green_navigator_node.py
|-- auv_telemetry/
|   |-- mission_logger_node.py      |-- mission_monitor_node.py  |-- trail_mapper_node.py
|-- auv_controls/                  [EMPTY STUB -- reserved for closed-loop cmd_vel PID]
|-- auv_propulsion/                [EMPTY STUB -- reserved for thruster allocation]
|-- auv_sensors/                  [EMPTY STUB -- reserved for EKF / real-hardware drivers]
|-- auv_msgs/                     [EMPTY STUB -- no custom interfaces defined; std msgs only]
'-- auv_teleop/                   [EMPTY STUB]
```

Fig 1. Workspace layout as built (10 packages; four are empty stub packages reserved for Phase 2).

Unlike the planned overactuated 8-thruster vehicle, the vehicle currently in the simulation is driven *kinematically*: Gazebo’s `gz-sim-velocity-control-system` plugin sets body velocity directly from `/model/auv_box/cmd_vel`, so `auv_controls` and `auv_propulsion` have nothing to run yet – this is a deliberate, documented placeholder (see Section 6), not an omission.

2 Salient Features of the Software Stack

- **Kinematic Velocity-Controlled Simulation** (`auv_description`): The AUV tracks `cmd_vel` exactly via Gazebo’s built-in `VelocityControl` plugin. A Fossen-model hydrodynamics plugin (BlueROV2 drag coefficients) is already attached to the hull but currently has *no effect* on motion – it activates automatically the moment the velocity-control plugin is swapped for individual thrusters in Phase 2.
- **Stereo-Vision Based 3D Localization** (`auv_vision`): Gate and green-mat 3D coordinates are computed by correlating YOLOv8n boxes (gate) / HSV contours (mat) with `stereo_image_proc` disparity maps. The gate localizer interrogates four prioritized candidate points across a 9×9 disparity patch (Left Pole → Right Pole → Crossbar → Center), takes the median of valid pixels, and applies $depth = f_x \cdot B / disparity$ with a plausibility guard (0.2 m–19.1 m).
- **Dual-Camera Cross-Handoff**: The front stereo pair handles long-range acquisition of the gate and green mat; the downward camera takes over for the blue-bin approach and fine centering, using odometry altitude for pinhole projection instead of stereo disparity.
- **Dead-Reckoning Visual Fallback** (`auv_planner`): Both navigators freeze the last verified target position in the odom frame and continue driving toward it when the live YOLO/HSV feed goes stale, then reconcile with fresh detections as they arrive.
- **HSV-Based False-Positive Reduction**: `blue_bin_cv_detector_node` (bottom camera) only accepts a blue blob if $\geq 15\%$ of its pixel area overlaps a dilated green-mat mask in the same frame – open blue pool water cannot pass this check, eliminating false positives with zero additional training data.
- **Advance-and-Sweep Search**: Rather than a fixed search radius, both the gate (lateral only) and green/blue-bin (lateral *plus* forward creep) searches strafe $\pm$ a bound, reverse at each limit, and – for the post-gate searches – creep forward and repeat if a full pass finds nothing, up to a bounded total-advance budget.
- **Post-Mission Trail Mapping & Live Dashboard** (`auv_telemetry`): `trail_mapper_node` records a pose whenever the AUV has moved ≥ 0.10 m, re-exports a colour-coded top-down PNG and an interactive 3D Plotly HTML map every 30 s and on shutdown, while `mission_monitor_node` renders a live ANSI terminal dashboard.
- **No active safety envelope yet**: tilt damping, a depth envelope, and wall repulsion are not implemented in the current controller – there is no standalone controller node; they are natural additions once `auv_controls` is built out.

3 Logical Steps in the Working of Software Components

Mission planning lives in `auv_planner`: `gate_navigator_v2_node` (the FSM actually launched by `sim_mission.launch.py`) for gate traversal, and `green_navigator_node` for everything after the gate.

3.1 Gate Navigation (`gate_navigator_v2_node`)

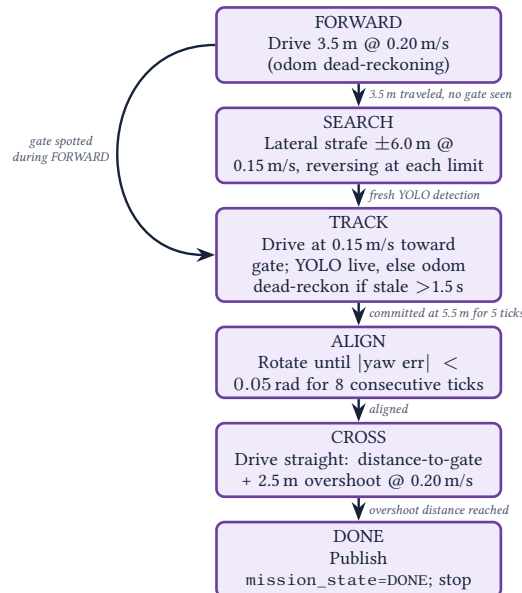


Fig 2. Gate Navigator State Machine (`auv_planner/gate_navigator_v2_node`) – values as configured in `sim_params.yaml`.

The legacy `gate_navigator_node` (round-trip crossing with a U-turn and return leg) is still present in the package but is *not* referenced by `sim_mission.launch.py`; switching to it is a one-line `executable=` change.

3.2 Post-Gate Navigation (`green_navigator_node`)

Once `gate_navigator_v2_node` publishes `DONE`, `green_navigator_node` runs an 11-state FSM – longer than a simple search/approach/centre pipeline because the mat and bin can sit well beyond camera range and the AUV must actively creep to find them:

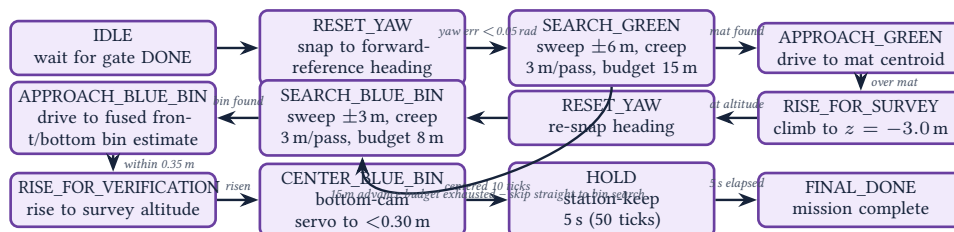


Fig 3. Post-Gate Navigator State Machine (`auv_planner/green_navigator_node`) – 11 states, values as configured in `sim_params.yaml`. There is no separate `CLEAR_GATE` surge state; `RESET_YAW` runs immediately on the gate's `DONE`.

Note

Two robustness details not visible in the diagram: the blue-bin odom estimate is fused from whichever camera is more reliable at the time (front-camera one-shot depth +1.0 m fudge until the bottom camera gets a confirmed-over-green reading), and `CENTER_BLUE_BIN` uses a leaky tick counter (drains by 1 on a bad tick instead of resetting to 0) so a single missed frame cannot restart the centering streak.

4 Dataset Preparation and Object Classes

4.1 Data Sources

Simulation-Generated Images: The competition arena is built in Gazebo Harmonic (PVC gate, 8 m×2 m green target mat, blue drum). `bottom_cam_altitude_test.py` is a standalone test-harness script (not a launched mission node) that teleports the AUV across a fixed grid of 5 altitudes ($z \in \{-2.0, -2.5, -3.0, -3.5, -4.0\}$ m) × 6 named horizontal offsets (over-bin, ±1 m left/right, +2 m right, 1 m forward, 1.5 m back) – 30 teleport poses per capture pass – saving annotated bottom-camera frames to `~/auv_ws/bottom_cam_test/`.

Kaggle and External Sources: Underwater imagery from Kaggle supplements the simulation captures with real turbidity, backscatter, and lighting variation.

4.2 Object Classes in the Dataset

The trained YOLOv8n weights (`gate_detection_v8.pt`, loaded by `gate_detector_node`) are multi-class – confirmed by the class-ID map used in the detector nodes and matching the confusion matrix in Fig. 4 – but **only two of those classes are currently wired into the ROS graph**: the red-bin and QR-code classes have no corresponding detector/localizer nodes yet.

Class	Geometry	Current ROS integration
Gate	Two vertical PVC posts + crossbar; camera_test.sdf posts at 3.0 m spacing	Live: YOLOv8n (class 3) + 4-candidate stereo disparity
Blue Bin	Cylindrical drum, 0.3 m radius, 0.5 m height, on the green mat	Live: YOLOv8n (class 1, front) + HSV green-overlap (bottom)
Red Bin	Same geometry as blue bin	Trained in the model (class 0) but no red_bin_detector / red_bin_localizer node exists in <code>auv_vision</code>
QR Code	Floor-mounted payload marker	Trained in the model (class 2) but no qr_detector / qr_localizer node exists
Green Target Mat	8 m × 2 m floor mat	Classical HSV segmentation (no YOLO class – by design)

Table: object classes trained vs. classes actually consumed by a running node today.

4.3 Model Details

YOLOv8n (nano) is the shared detection backbone for both the gate and blue-bin detectors. The green mat uses classical HSV thresholding with morphological cleanup exclusively – no neural inference is run for it. Fig. 4 and Fig. 5 below are the training artifacts for the currently-loaded weights and are unchanged from the underlying training run.

Six checkpoints are retained side-by-side in `auv_vision/weights/` (`gate_detection_v2/v4/v5/v6/v7/v8.pt`), spanning class-ID remaps as the label set grew – `gate_detector_node` loads `v8.pt` (gate = class 3) while `blue_bin_detector_node` loads the earlier `v7.pt` (blue bin = class 1); the two detector nodes are pinned to different checkpoints rather than sharing one, so a future retrain has to keep both class maps in mind or update both `model_weights` parameters together.

Live Detection Capture (a byproduct, not a re-training pipeline)

Every detector node also writes debug frames during normal operation, which incidentally accumulates more real candidate training data as missions are run: `gate_detector_node` overwrites a single `latest_detection.jpg` in `~/ros/yolo_debug/`; `blue_bin_detector_node` keeps every individual detection, numbered and confidence-tagged, in a fresh `~/ros/blue_bin_detections/run_<timestamp>/` folder per launch; `blue_bin_cv_detector_node` and `green_detector_node` each overwrite one annotated frame showing their HSV mask overlay. None of this feeds an automated re-training loop today – it exists purely for live debugging.

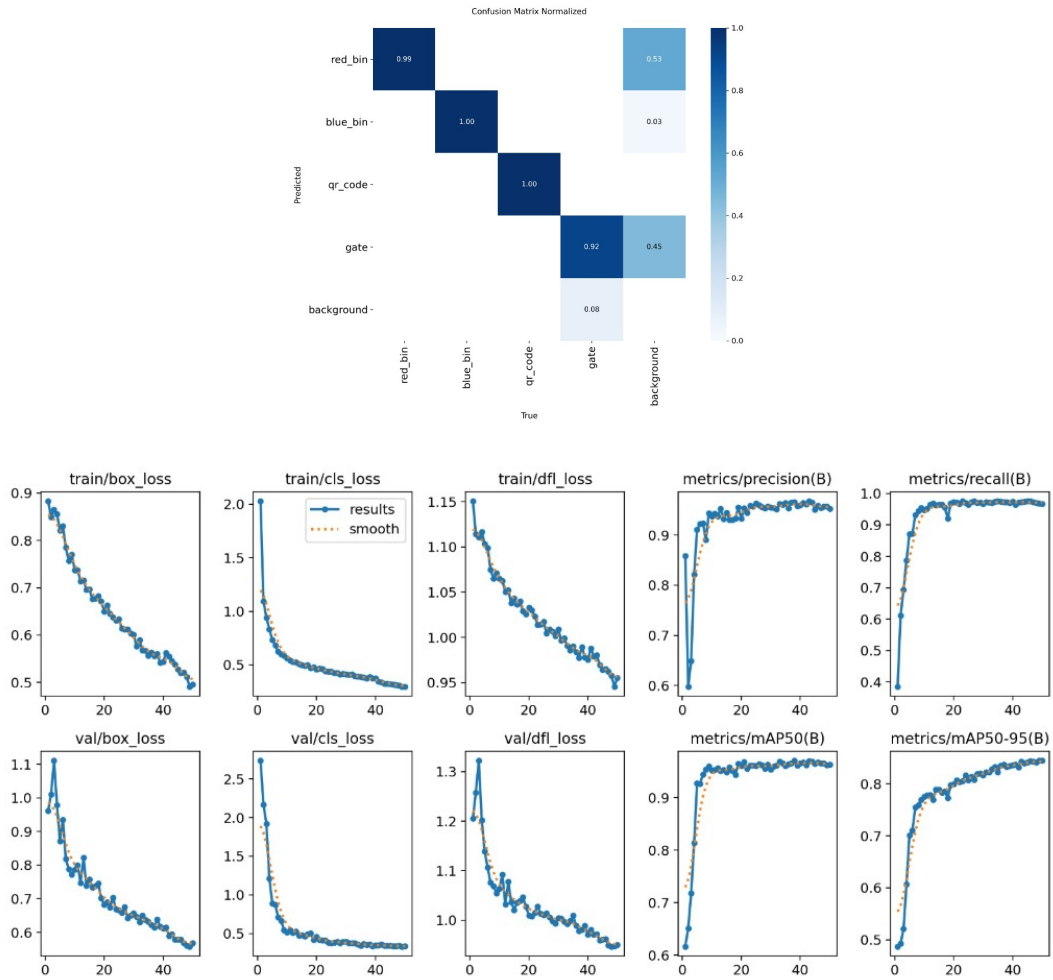


Fig 4/5. YOLOv8n training confusion matrix and loss/precision/recall/mAP curves for the weights currently loaded by gate_detector_node and blue_bin_detector_node.

5 Object Detection and Vision Pipeline

5.1 Gate Detection Pipeline

`gate_detector_node` subscribes to `/model/auv_box/stereo_front/left/image_raw`, runs YOLOv8n at confidence 0.10, and publishes the single highest-confidence box to `/auv/gate_detection_2d`. `gate_localizer_node` fuses this with `stereo_image_proc` disparity: it samples a 9×9 patch at each of Left Pole, Right Pole, Crossbar, Center (in that priority order), accepts the first with ≥ 3 valid pixels, and computes $Z = f_x B/d$, $X = (u - c_x)Z/f_x$, $Y = (v - c_y)Z/f_y$ (baseline $B = 0.075$ m as configured in `sim_params.yaml`), guarded to 0.2–19.1 m, publishing `[x,y,z,conf]` to `/auv/gate_position_3d`.

5.2 Blue Bin Detection Pipeline

`blue_bin_detector_node` runs YOLOv8n (class 1) on *both* the front and bottom camera feeds ($\text{conf} \geq 0.70$), publishing to `/auv/blue_bin_detection_{front,bottom}_2d`. `blue_bin_cv_detector_node` replaces the bottom-camera YOLO path with an HSV blue/green overlap test ($\geq 15\%$ overlap with a dilated green mask) on the same topic, so open water cannot false-positive. `blue_bin_localizer_node` prefers the bottom-camera altitude projection when a confident bottom detection exists, else falls back to a front-camera stereo slant-range estimate.

5.3 Green Mat Detection Pipeline

`green_detector_node` applies HSV segmentation ($H:38-82$, $S:80-255$, $V:60-255$). On the front camera it only processes the bottom 40% of the frame (`front_roi_top_frac=0.60`), since the mat only ever appears low in that horizontally-mounted camera's field of view; the bottom camera processes the full frame and reports a coverage fraction used as the Phase-1/Phase-2 handoff signal in `green_localizer_node`.

5.4 Underwater View Rendering

`underwater_view_node` applies Beer-Lambert attenuation ($I_\lambda = e^{-\alpha_\lambda r} I_{0,\lambda} + (1 - e^{-\alpha_\lambda r}) \text{bg}_\lambda$) to the left camera feed using

the disparity-derived range, publishing a separate, display-only `/auv/underwater_view/image_raw` topic – every detector still reads the raw, un-hazed feed. This exists because Gazebo Harmonic’s `<fog>` tag is a no-op and the world’s water volume cannot tint a camera sitting inside it.

Note

Not yet implemented: `red_bin_detector_node`, `red_bin_localizer_node`, `qr_detector_node`, `qr_localizer_node`. The trained model already supports the classes; only the ROS wiring is missing.

6 Controls, Propulsion & Thruster Allocation

This section differs the most from the vehicle’s eventual design. There is currently no closed-loop velocity controller node and no thruster-allocation node in the software stack – `auv_controls` and `auv_propulsion` are empty package stubs (`CMakeLists.txt` + `package.xml` only). Motion is produced directly by Gazebo:

- **gz-sim-velocity-control-system** reads `/model/auv_box/cmd_vel` (a `geometry_msgs/Twist` published by whichever navigator is active) and sets the model’s body velocity *directly*, every physics step – perfect velocity tracking, no PID, no thruster forces.
- **gz-sim-hydrodynamics-system** is attached to `base_link` with Fossen-model added-mass and quadratic-drag coefficients taken from the published BlueROV2 values ($X_{u|u}=-33.7$, $Y_{v|v}=-54.2$, $Z_{w|w}=-73.2$, roll/pitch/yaw drag $=-4.0$). These coefficients currently have **zero effect** on motion, because `VelocityControl` overwrites the resulting forces every step – they exist pre-wired so that removing the velocity-control plugin is the only change needed to make drag real.
- The eight thruster meshes (`Thruster_1.STL` – `Thruster_8.STL`) are present visually on the model but have **no joints or links** in `model.sdf` – they render but do not actuate.

Documented Phase-2 Path (from `model.sdf` comments)

(1) Remove the `VelocityControl` plugin. (2) Add one `gz-sim-thruster-system` per thruster, joined to the existing meshes. (3) Implement Moore-Penrose thrust allocation in `auv_propulsion`. (4) Implement a closed-loop `cmd_vel` PID in `auv_controls` so the navigators’ existing `Twist` interface needs no changes. Only after step 1 do the hydrodynamics coefficients above start affecting the vehicle.

No tilt-damping, depth-envelope, or wall-repulsion safety logic exists anywhere in the current stack; these are natural additions to the not-yet-built `auv_controls` node.

7 Sensor Suite, State Estimation & Sim-to-Real Portability

7.1 Sensor Suite & Camera Specifications

Sensor	Topic	Rate / Res.	Role
Stereo Front Pair	<code>.../stereo_front/{L,R}/image_raw</code>	15 Hz 640×480, HFOV 73°	Gate & green-mat long-range 3D localization
Downward Camera	<code>.../bottom/image_raw</code>	15 Hz 640×480, HFOV 73°	Blue-bin/green-mat close-range tracking and centering
Ground-truth Odometry	<code>/auv/odom</code>	50 Hz	Position, orientation, twist – consumed by every navigator, localizer, and logger

There is **no IMU and no separate depth sensor** modeled in `model.sdf` today – depth comes from the Z channel of the ground-truth odometry published by `gz-sim-odometry-publisher-system` (`odom_frame=map`, `robot_base_frame=base_link`, 50 Hz).

7.2 State Estimation

`auv_sensors` is an empty stub package: there is currently **no Extended Kalman Filter and no robot_localization node running**. The stack relies entirely on Gazebo’s ground-truth odometry. `real_params.yaml` already anticipates the switch – it sets `trail_mapper_node.odom_frame: 'odom'` with a comment noting that real hardware will source it from a `robot_localization` EKF – but no launch file or EKF config exists yet to consume it.

7.3 Sim-to-Real Configuration Isolation

Tunable node parameters are isolated in `auv_bringup/config/:sim_params.yaml` (loaded by `sim_mission.launch.py`) and `real_params.yaml` (parameter values only, pre-written for hardware but not yet wired to a hardware launch file – there is no `mission.launch.py` in the workspace yet). Node source code is unchanged between the two; only YAML values differ (e.g. `cross_speed` drops from 0.20 to 0.15 m/s, and `track_speed/search_spin_speed` are lowered for a conservative first real run).

8 Software Used

Component	Purpose in this stack
ROS 2 Jazzy	Core middleware – node lifecycle, DDS pub/sub, launch, parameters.
Ultralytics YOLOv8n	Gate and blue-bin bounding-box detection.
OpenCV (<code>cv_bridge</code>)	Image conversion, HSV thresholding/contours, Beer-Lambert haze rendering.
Gazebo Harmonic	Physics simulation; <code>VelocityControl</code> kinematic drive + inert Hydrodynamics plugin.
<code>stereo_image_proc</code>	Stereo rectification and disparity generation from the front camera pair.
<code>ros_gz_bridge</code> / <code>ros_gz_image</code>	Bridges Gazebo image, odometry, and <code>cmd_vel</code> topics into ROS 2.
Matplotlib / Plotly	Post-mission 2D PNG and interactive 3D HTML trajectory maps.
Python 3	Every custom node is pure Python.

Not currently in the runtime stack, despite being anticipated in comments/config: `robot_localization` (EKF) and `gz-sim-thruster-system` (per-thruster physics).

9 Sensor and Communication Circuit (Software Topology)

Raw optical streams from the front stereo pair and bottom camera split into YOLO/HSV detection paths and the disparity depth path, merging at the localizer nodes to produce 3D coordinates. The two diagrams below are drawn directly from the subscriptions/publications declared in each node’s source – not from the aspirational pipeline in earlier drafts of this document.

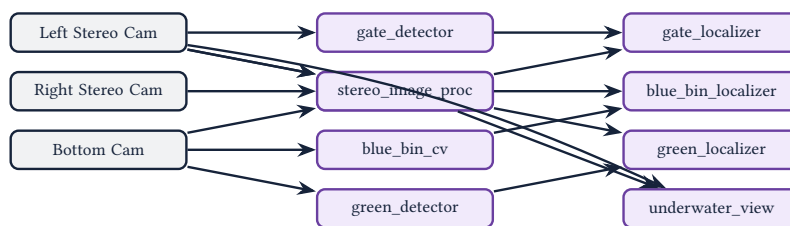


Fig 6. Perception Pipeline Topology (`auv_vision`) – as built: no red-bin or QR branches exist; `blue_bin_localizer` fuses two independent bottom-camera detectors (YOLO and HSV) with the front-camera YOLO path.

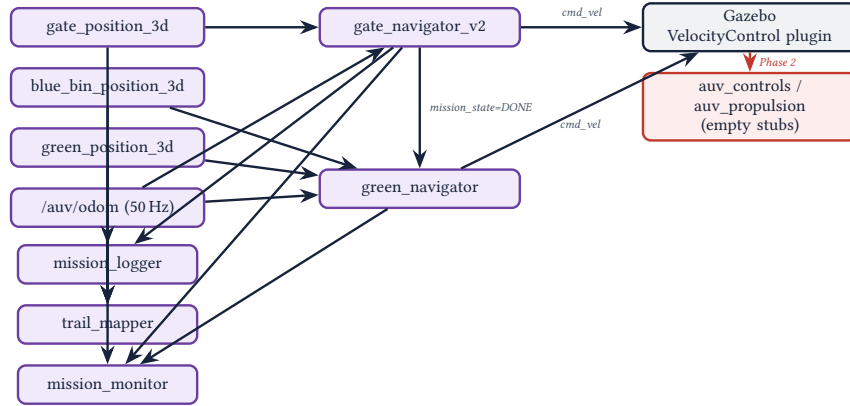


Fig 7. Planning, Motion & Telemetry Topology – `cmd_vel` drives the Gazebo velocity-control plugin directly; there is no velocity_controller or thruster_allocator node, and no wrench_cmd/cmd_thrust chain.

9.1 Comprehensive ROS 2 Topic Catalog (as built)

Topic	Publisher	Type	Role
.../stereo_front/left/image_raw	ros_gz_bridge	sensor_msgs/Image	Left stereo feed – gate/blue-bin YOLO, stereo_image_proc
.../bottom/image_raw	ros_gz_bridge	sensor_msgs/Image	Downward feed – blue-bin CV, green HSV
/disparity	stereo_image_proc	stereo_msgs/DisparityImage	Dense disparity for gate/green localizers
/auv/gate_detection_2d	gate_detector	Float32MultiArray	Gate bbox [det,cx,cy,w,h,conf]
/auv/gate_position_3d	gate_localizer	Float32MultiArray	[x,y,z,conf] body frame
/auv/blue_bin_detection_front_2d	blue_bin_detector	Float32MultiArray	Front-camera YOLO 2D bin detection
/auv/blue_bin_detection_bottom_2d	blue_bin_detector, blue_bin_cv_detector	Float32MultiArray	Two publishers on one topic: bottom-cam YOLO and HSV overlap detection
/auv/blue_bin_position_3d	blue_bin_localizer	Float32MultiArray	[x,y,z,conf,source] fused estimate
/auv/green_detection_{front,bottom}_2d	green_detector	Float32MultiArray	Per-camera 2D mat detection + coverage
/auv/green_position_3d	green_localizer	Float32MultiArray	[x,y,z,conf,source]
/auv/mission_state	gate_navigator_v2	std_msgs/String	FORWARD/SEARCH/TRACK/ALIGN/CROSS/DONE
/auv/green_mission_state	green_navigator	std_msgs/String	11-state post-gate FSM
/auv/forward_reference_yaw	gate_navigator_v2	std_msgs/Float32	Mission-start heading, for RESET_YAW
/model/auv_box/cmd_vel	active navigator	geometry_msgs/Twist	Consumed directly by Gazebo VelocityControl
/auv/odom	ros_gz_bridge (remapped)	nav_msgs/Odometry	Ground-truth pose/twist, 50 Hz
/auv/trail_map	trail_mapper	nav_msgs/Path	Full trajectory, 1 Hz
/auv/object_markers	trail_mapper	visualization_msgs/MarkerArray	Gate cuboid + bin sphere, for RViz2
/auv/underwater_view/image_raw	underwater_view	sensor_msgs/Image	Display-only hazed feed

Table: active topics as wired today. `/auv/qr_payload`, red-bin position topics, `/auv/wrench_cmd`, and `cmd_thrust` do not exist in the current stack.

10 Mission Telemetry & Mapping

`trail_mapper_node` appends a pose whenever the AUV has moved ≥ 0.10 m, publishes the accumulated `nav_msgs/Path` at 1 Hz, and re-exports a top-down PNG map plus an interactive 3D Plotly HTML reconstruction every 30 s and on shutdown (atomic write-then-rename, so a forced kill mid-save cannot corrupt the output file). `mission_logger_node` writes a 26-column CSV of gate-phase telemetry at 1 Hz, and `mission_monitor_node` renders a live ANSI dashboard covering both FSMs.

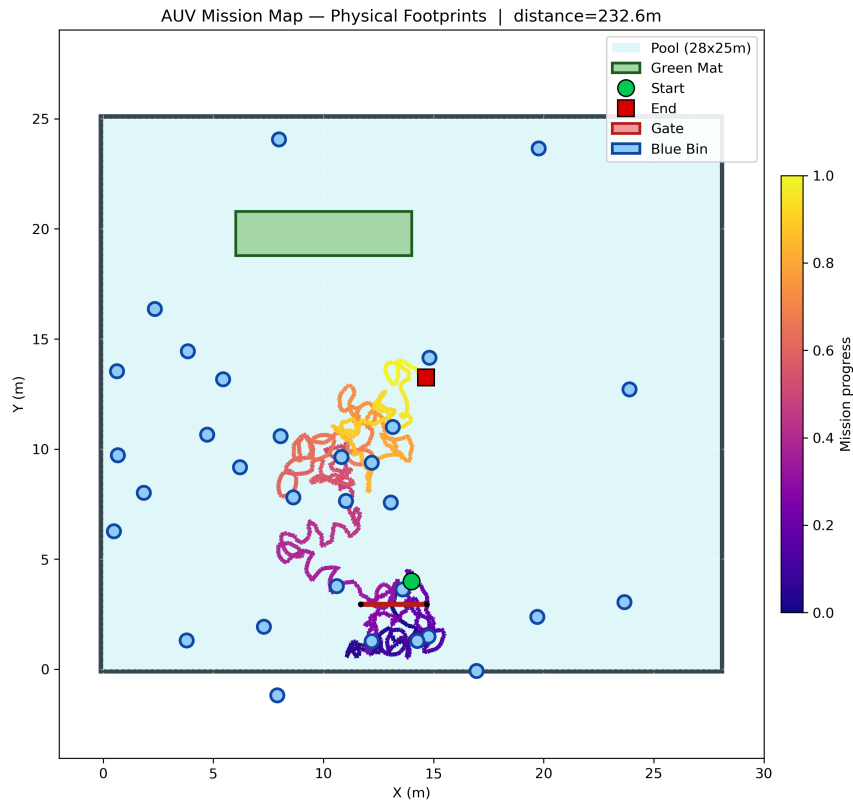


Fig 8. Most Recent Mission Trajectory Map (map_20260828_123328 .png), regenerated from the latest logged run. The trail (yellow=late, purple=early) shows a gate crossing followed by an extended green-mat/blue-bin search phase; the scattered blue-bin markers off the mat reflect transient bottom-camera false detections from that specific run, each independently tracked by `trail_mapper_node`'s per-object association logic rather than a single clean bin fix.

This replaces the earlier 45.3 m single-pass map: the current run covers 232.6 m because the post-gate search exercised the full advance-and-sweep budget before converging on the green mat, which is exactly the behaviour Section 3.2 describes.

11 Summary: What Changed From the Prior Documentation

Area	Previously documented	As built today
Propulsion	8-thruster Moore-Penrose allocation, closed-loop wrench controller	Kinematic <code>VelocityControl</code> plugin; <code>auv_controls</code> / <code>auv_propulsion</code> empty
Safety envelope	Tilt damping, depth envelope, wall repulsion	Not implemented anywhere in the stack
State estimation	EKF fusing IMU + depth via <code>robot_localization</code>	Ground-truth Gazebo odometry only; no IMU modeled, no EKF running
Gate TRACK→ALIGN	Commits at a 1.5 m staging point	Commits at 5.5 m, held for 5 ticks
Gate CROSS overshoot	3.25 m	2.5 m
Post-gate FSM	8 linear stages incl. CLEAR_GATE surge & coverage-based rise	11-state FSM with advance-and-sweep search and a fixed-altitude survey rise; no CLEAR_GATE state
Red bin / QR	Dedicated detector + localizer nodes	Classes exist in the trained model; no ROS nodes implemented
Blue-bin centering	± 20 px image-space tolerance	0.30 m metric tolerance via bottom-camera servoing

Table: the material differences between the original architecture description and the code in this workspace today.